

确定何时需要检索增强，以提升效率并避免干扰信息；**(2) 何处增强**，讨论生成过程中插入检索信息的常见位置；**(3) 多次增强**，针对复杂与模糊查询，讨论常见的多次增强方式；**(4) 降本增效**，介绍现有的知识压缩和缓存加速策略。详细内容将在 6.4 节介绍。

本节初步介绍了**我们为什么需要 RAG** 和 **RAG 是什么** 这两个问题。接下来的章节将针对上面提出的三个问题，对具体技术细节详细讨论。

6.2 检索增强生成架构

检索增强生成 (RAG) 系统是一个集成了外部知识库、检索器、生成器等多个功能模块的软件系统。针对不同的业务场景和需求，可以设计不同的系统架构来组合、协调这些模块，以优化 RAG 的性能。其中，检索器和生成器的协作方式对 RAG 性能的影响最为显著。这是因为在不同的协作方式下，检索器检索到的信息质量会有所不同，生成器生成的内容质量也会随之变化。此外，检索器和生成器之间的协作方式对系统的效率有很大影响。高效的协作能够减少延迟，提高系统的响应速度。本节将从**如何优化检索器与大语言模型的协作**这一角度出发，对经典 RAG 架构进行梳理和介绍。

6.2.1 RAG 架构分类

针对不同的业务场景，RAG 中的生成器可以选用不同的大语言模型，如 GPT-4[1]、LLaMA[49] 等。考虑到大语言模型的开源/闭源、微调成本等问题，RAG 中的大语言模型可以是参数不可感知/调节的“黑盒”模型，也可以是参数可感知和微调的“白盒”模型。例如，如果选用 GPT-4，由于其闭源性，在 RAG 过程中只能将其视为“黑盒”，只能利用其输出结果，而无法感知/微调其模型参数。如果选择

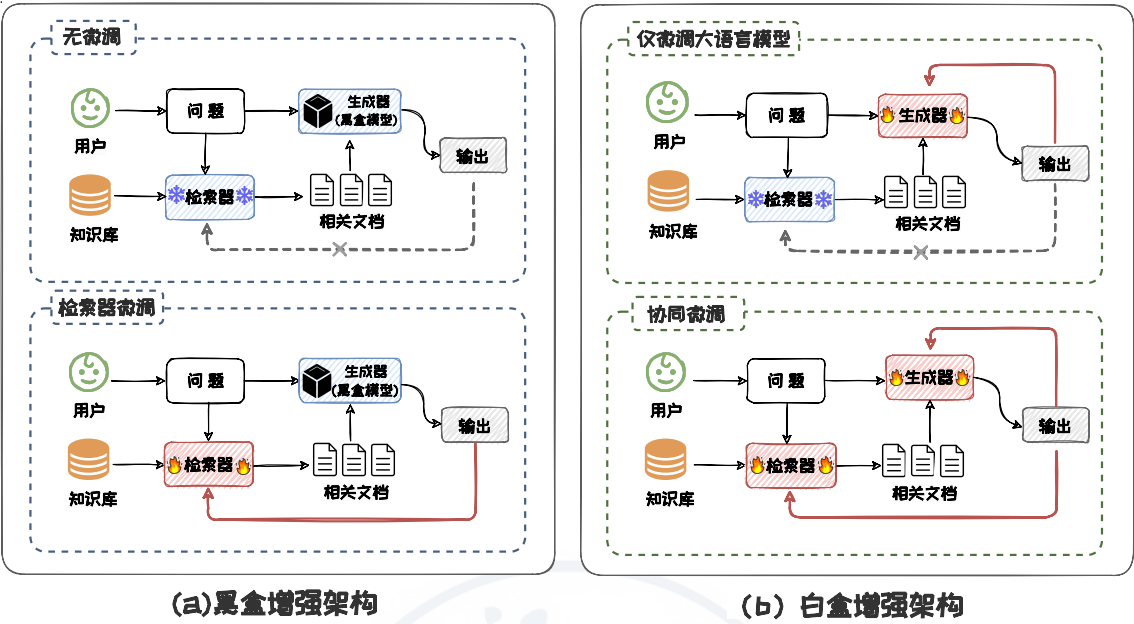


图 6.9: 检索增强架构分类图。其中含蓝色雪花的模块表示其参数被冻结、带红色火焰的部分表示微调时其参数被更新。

LLaMA 模型，在计算资源允许的情况下，在 RAG 过程中可将其视为“白盒”并对其进行微调。从是否对大语言模型进行微调的角度出发，本小节将 RAG 架构分类两大类：**黑盒增强架构**和**白盒增强架构**，如图6.9所示。

其中，黑盒增强架构可根据是否对检索器进行微调分为两类：**无微调**、**检索器微调**，如图6.9(a)所示。在无微调架构中，检索器和大语言模型都不进行任何微调，仅依靠它们在预训练阶段掌握的能力完成相应的检索和生成任务。在检索器微调的架构中，语言模型参数保持不变，而检索器根据语言模型的输出反馈进行参数的针对性调整。类似的，白盒增强架构也可根据是否对检索器进行微调分为两类：**仅微调大语言模型**、**检索器与大语言模型协同微调**（下文简称为协同微调），如图6.9(b)所示。在仅微调大语言模型的架构中，检索器作为一个预先训练好的组件其参数保持不变；语言模型则根据检索器提供的相关信息进行参数调整。在协同微调的架构中，检索器和大语言模型迭代交互、协同微调。

在 RAG 系统中，除了调整检索器和大语言模型，我们也可对其他功能模块（如知识库中的向量 [17, 45]）进行调整。调整其他功能模块与黑盒增强和白盒增强的分类是兼容的。本节接下来的部分将详细介绍黑盒增强架构和白盒增强架构，并探讨它们代表性方法。

6.2.2 黑盒增强架构

在某些情况下，由于无法获取大语言模型的结构和参数或者没有足够的算力对模型进行微调，例如只能通过 API 进行交互时，我们不得不将语言模型视为一个黑盒。此时，RAG 需要在黑盒增强架构的基础上构建。在黑盒增强架构中，我们仅可对检索器进行策略调整与优化。其可以分为无微调架构和检索器微调两种架构。接下来对两种架构类型分别展开介绍。

1. 无微调

无微调架构是所有 RAG 架构中形式最简单的。该架构中，检索器和语言模型经过分别独立的预训练后**参数不再更新，直接组合使用**。这种架构对计算资源需求较低，方便实现且易于部署，适合于对部署速度和灵活性有较高要求的场景。In-Context RALM[42] 是该框架下的代表性方法。其直接将检索器检索到的文档前置到输入问题前作为上下文，方法示意图如图6.10所示。In-Context RALM 包括检索和生成两个阶段。在检索阶段，输入的问题或部分句子作为查询从知识库中检索出相关文档。在生成阶段，这些检索到的文档被直接拼接到 Prompt 中的上下文部分，然后将 Prompt 输入给大语言模型。一个 RAG 任务可能涉及多次执行检索和生成。例如，在一个长文本生成任务中，每生成一定量的文本后，模型就可能会执行一次检索，以确保随着话题的发展，后续生成的内容能够持续保持与话题相关。

在执行检索操作时，需要仔细选择几个关键参数，如**检索步长**和**检索查询长度**。检索步长是指模型在生成文本时，每隔多少个词进行一次检索，这一参数的设

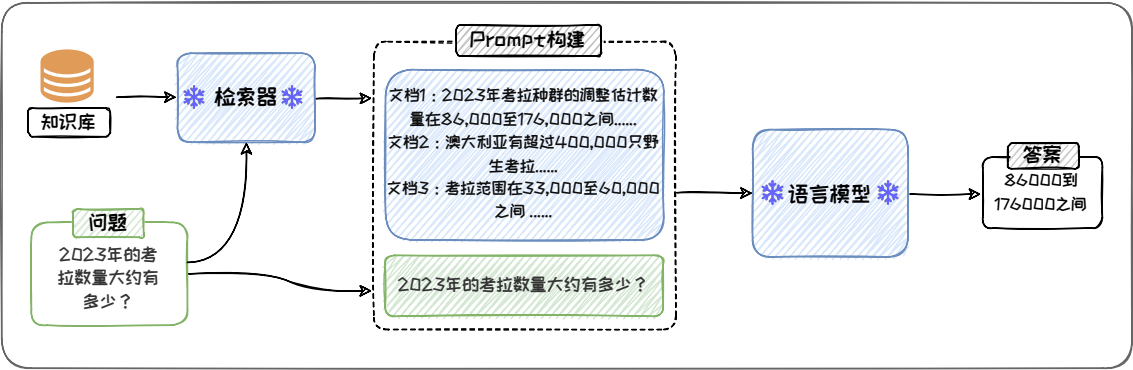


图 6.10: In-Context RALM 模型架构图。

定直接影响到模型的响应速度和信息的即时性。较短的检索步长能够提供更为及时的信息更新，但同时也可能增加计算的复杂性和资源消耗。因此，在实际应用中，需要在这两者之间找到一个合理的平衡点。检索查询长度指的是用于检索的文本片段的长度，通常被设置为语言模型输入中的最后几个词，以确保检索到的信息与当前的文本生成任务高度相关。

2. 检索器微调

虽然无微调架构在实现和部署上非常便捷，但它完全没有考虑检索器与语言模型之间潜在的协同效应，效果有待提升。为了进一步提升效果，可以采用检索器微调架构对检索器进行微调，以更好地适用于黑盒增强的环境。在检索器微调架构中，大语言模型的**参数保持不变**，仅用其输出指导检索器的微调。这种架构下的检索器能更好地适应大语言模型的需求，从而提高 RAG 的表现。

REPLUG LSR[45] 是检索器微调框架的代表性方法，其结构如图6.11所示。它使用大语言模型的困惑度分数作为监督信号来微调检索器，使其能更有效地检索出能够显著降低语言模型困惑度的文档。其微调检索器的过程中采用 KL 散度损失函数来训练检索器，目的是对齐检索到的文档的相关性分布与这些文档对语言模型性能提升的贡献分布。此过程涉及两个关键的概率分布，第一个是**检索器输出的文档分布**：检索器在接收到当前上下文后检索与之相关的文档，并形成一

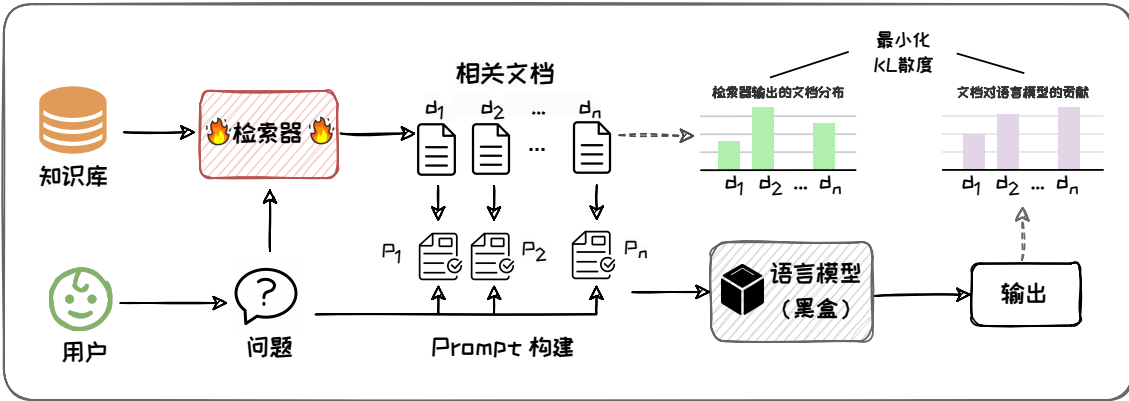


图 6.11: REPLUG LSR 模型架构图。

档概率分布。这一分布是基于检索器计算的上下文与文档之间的相似度，通过余弦相似度来衡量，并将这些相似度分数转化为概率值。第二个是**文档对语言模型的贡献分布**：语言模型为每个被检索到的文档和原始上下文来生成预测，最终所有输出结果形成一个概率分布。在这个分布中，如果某个文档对语言模型生成准确预测特别关键，它会被赋予更高的概率权重。在微调过程中，REPLUG LSR 将语言模型视为黑盒处理，仅通过模型的输出来指导检索器的训练，避免了对语言模型内部结构的访问和修改。此外，在微调过程中，REPLUG LSR 还采用了一种异步索引更新策略，即不会在每次训练步骤后立即更新知识库的向量编码，而是在一定的训练步骤之后才进行更新。这种策略降低了索引更新的频率，减少了计算成本，使模型能够在连续训练过程中更好地适应新数据。此外，检索器微调框架中还可以引入代理模型来指引检索器微调。例如，AAR[60]方法通入引入额外的小型语言模型，使用它的交叉注意力得分标注偏好文档，以此来微调检索器，使其能够在不微调目标语言模型的情况下增强其在不同任务上的表现。

检索器微调的方式允许即使是闭源大模型如 ChatGPT，也能通过优化外部检索器来提升性能。REPLUG LSR 和 AAR 通过该方式实现了在保持大模型完整性的同时，通过外部调整来增强模型的能力，这在传统的 RAG 中是不常见的。

6.2.3 白盒增强架构

通常，大语言模型和检索器是独立预训练的，二者可能存在匹配欠佳的情况。白盒增强架构通过微调大语言模型来配合检索器，以提升 RAG 的效果。其可根据是否对检索器进行微调分为两类：**仅语言模型微调**、**检索器和语言模型协同微调**。

1. 仅微调语言模型

仅微调语言模型指的是检索器作为一个预先训练好的组件其参数保持不变，大语言模型根据检索器提供的上下文信息，对**自身参数进行微调**。RETRO[5] 是仅微调语言模型的代表性方法之一。该方法通过修改语言模型的结构，使其在微调过程中能够将从知识库中检索到的文本直接融入到语言模型中间状态中，从而实现外部知识对大语言模型的增强。此外，SELF-RAG[3] 通过在微调语言模型时引入反思标记，使语言模型在生成过程中动态决定是否需要检索外部文本，并对生成结果进行自我批判和优化。这些方法不仅提高了生成内容的质量和事实准确性，还增强了模型的知识整合与应用能力。

以 RETRO 为例，其结构如图6.12所示。RETRO 首先将知识库中的文本进行切块，然后用 BERT 对每个文本块生成嵌入向量。在微调模型时的自回归过程中，每当模型生成一段文本块后，就去知识库中检索出与之最相似的嵌入向量。然后，这些嵌入向量和模型注意力层的输出一起被送入一个外部的 Transformer 编码器进行编码。得到的编码向量直接输入给模型的块交叉编码器的键（key）和值（value），以捕捉外部知识的关键信息。通过交叉编码，模型能够结合检索到的相关信息来生成新的文本块。

通过上述方式微调后的 RETRO 模型能够充分整合检索到的信息，生成连贯且富含信息的文本。面对用户查询时，模型能展现出优秀的理解能力和知识整合能力，大幅提升了生成的质量和准确性，尤其在处理复杂任务时，其表现更为突出。

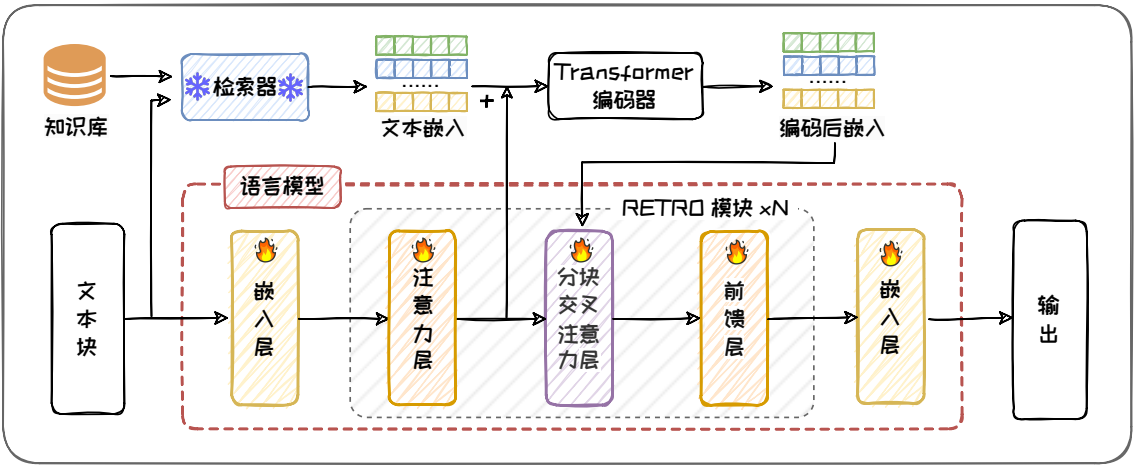


图 6.12: RETRO 模型架构图。

2. 检索器和语言模型协同微调

在仅微调语言模型的架构下，检索器作为固定组件，微调过程中其参数保持不变。这导致检索器无法根据语言模型的需求进行适应性调整，从而限制了检索器与语言模型之间的相互协同。在检索器和语言模型协同微调的架构中，检索器和语言模型的参数更新同步进行。这种微调的方式使得检索器能够在检索的同时学习如何更有效地支持语言模型的需求，而语言模型则可以更好地适应并利用检索到的信息，以进一步提升 RAG 的性能。

Atlas[17] 是该架构的代表性工作，其架构如图 6.13 所示。与 REPLUG LSR 类似，其在预训练和微调阶段使用 KL 散度损失函数来联合训练检索器和语言模型，以确保检索器输出的文档相关性分布与文档对语言模型的贡献分布相一致。不同之处在于，Atlas 在预训练和微调过程中，检索器和语言模型参数同步被更新，检索器学习向语言模型提供最相关的文档，而语言模型则学习如何利用这些文档来改善其对查询的响应。为了确保检索结果与模型最新状态保持同步，Atlas 同样需要定期更新语料库文档的向量编码，从而维持检索的准确性。

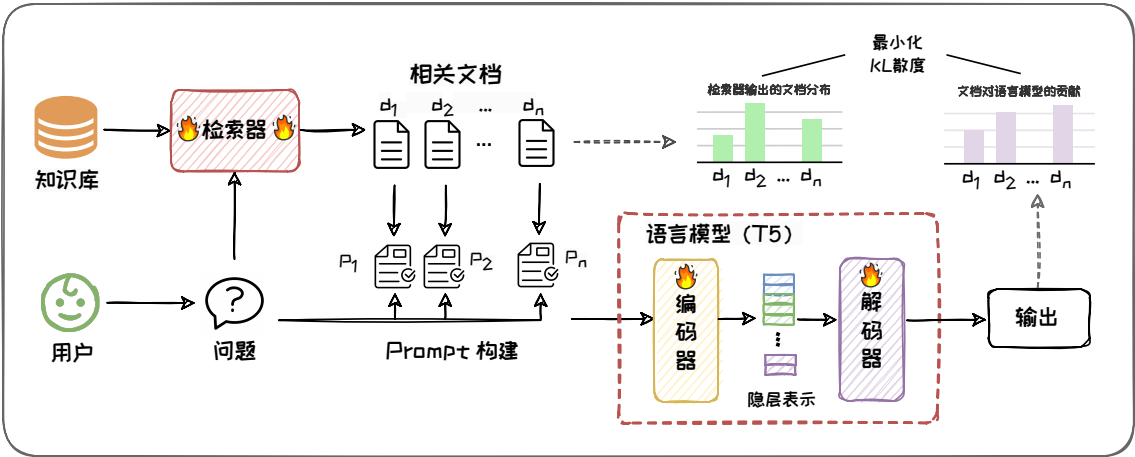


图 6.13: Atlas 模型架构图。

6.2.4 对比与分析

本节主要介绍了 RAG 的黑盒增强架构和白盒增强架构及其经典方法。接下来，我们对这两种架构进行总结和对比。

黑盒增强架构是在闭源模型的背景下提出的，它限制了对模型内部参数的直接调整。在这种架构下，我们介绍了**无微调**和**检索器微调**两种策略。**无微调**简单实用，它直接利用预训练的语言模型和检索器，不进行任何更新，适合快速部署。然而，这种方法的缺点在于无法对语言模型进行优化以适应新的任务需求。相比之下，**检索器微调**通过调整检索器来适应语言模型输出，提供了在无法修改语言模型的情况下提升性能的可能性。这种方法的效果在很大程度上取决于调整后的检索器的准确性。

白盒增强架构则利用开源模型的优势，允许调整语言模型结构和参数，可以更好的协调减速器和大语言模型。在这种架构中，我们介绍了两种微调形式：**仅微调语言模型**和**检索器和语言模型协同微调**。**仅微调语言模型**专注于优化语言模型，根据检索到的信息仅调整语言模型结构和参数，以提升特定任务上的性能。**检索器和语言模型协同微调**是一种更为动态的策略，它通过同步更新检索器和语言模